OpenBench: Trusted Sharing of Underused Spaces & Equipment

Hackathon: 48-Hour Build | **Problem statement:** Unlock Underused Equipment and Spaces **Team name:** [TEAM NAME] | **Live product:** [PUBLIC URL] | **Source code:** [GITHUB URL] (access: public / judges invited)

---

1. Selected use case

One institution, many owners. A fictional engineering college, *Nalanda Institute of Technology*, whose departments own valuable but idle resources: a laser cutter, 3D printers and CNC router (Fab Lab), a GPU workstation (Computing Lab), a camera kit and podcast studio (Media Centre), an oscilloscope bench (Electronics Lab) and a seminar room. These are shared with verified students, researchers, startup teams and community groups nearby.

We chose a single-institution scope deliberately: the problem statement prefers "a reliable solution for a specific setting" over "an unfinished platform for everyone". The data model (owners, managers, resources, member types) already supports multiple organisations.

2. Target users

User	Need	What OpenBench gives them
Student / researcher / startup / community member	Access to equipment they cannot afford, and a clear answer on *why* they were approved, rejected or waitlisted	Smart search, eligibility checklist, live request preview with fairness score, waiting list with automatic promotion, full booking history
Resource owner (department head)	Keep control of expensive property; know who, why, when, and in what condition	Defines conditions per resource, reviews every high-risk request, deposits, condition records, incident handling
Lab manager / technician	Day-to-day operation: supervise, check users in, confirm returns	Review queue ordered by explainable priority, on-site check-in, return confirmation, incident resolution
Administrator	Trust and fairness across the institution	Identity verification, fair-use watch, independent appeals, tamper-evident audit trail

3. Why the problem matters (reasoning and evidence)

- **Idle assets:** teaching labs, makerspaces and studios are typically used only during timetabled hours and sit idle in evenings, weekends and vacations, while outsiders and other departments cannot access them.

[ADD: one real source or a short interview/survey result from your own research, e.g. a campus utilisation study or a quote from a lab technician.]

- **Why ordinary booking tools fail:** a calendar answers "is it free?", not "is this person trained, who is responsible, what condition was it in, what happens if it breaks, and was allocation fair?". Without those controls owners rationally say "no", and the equipment stays idle.
- **Failure modes we designed against:** unqualified use of dangerous equipment (laser, CNC), disputes over damage ("it was already scratched"), no-shows blocking scarce slots, one heavy user monopolising a GPU, unexplained rejections, and unreviewable automated penalties.

4. Proposed solution

OpenBench turns "sharing" from a favour into a controlled process. Four design principles:

1. **Rules before requests** - owners define training, member types, deposits, supervision, limits and opening hours; the system checks them *before* a request is sent and tells the user exactly what is missing.
2. **Humans decide, the system explains** - approvals, rejections, waitlisting, penalties and deposits are human decisions with mandatory written reasons. Automation is limited to low-risk auto-approval (opt-in per resource), reminders, flagging and ordering.
3. **Fairness is explicit** - per-person weekly quotas and open-request caps, plus a transparent priority score that orders competing requests.
4. **Everything is on the record** - a hash-chained audit trail shared (per booking) with both parties.

5. Complete product workflow (maps to the 10 challenge steps)

#	Challenge step	In OpenBench
1	Owner lists a resource and defines conditions	*Manage > Resources > List a new resource*: training, member types, deposit, supervised, auto-approve, high-demand, limits, opening hours
2	User discovers it and checks eligibility	*Discover*: keyword + intent matching ("engrave acrylic") with match explanations; resource page shows a live eligibility checklist
3	User submits a booking request	Date/time/purpose form with **Check my request** preview (predicted outcome + fairness breakdown)
4	System verifies requirements	Server re-validates identity, standing, member type, certification, opening hours, max duration, weekly quota, open-request cap, overlaps. Failures return plain-language reasons
5	Owner/manager reviews	Review queue sorted by priority; competing requests listed; requester shown in a privacy-limited view
6	Approve / reject / waitlist	One click, written reason required for reject/waitlist; reasons shown to the user. Double-booking is impossible (409). If an approved booking is cancelled or no-show, waitlisted requests are promoted back to review in priority order
7	Check-in with initial condition	User (or staff on supervised equipment) records condition + notes; early check-in allowed in demo mode

#	Challenge step	In OpenBench
8	Return with final condition	Condition compared with check-in; owner confirms return and releases or withholds the (simulated) deposit with a reason
9	Delay / damage / conflict reported and handled	Auto-flags for worse condition, late return (>15 min), no-show, overdue; manual incident reports; manager resolves with a human-chosen penalty; user can appeal to an admin
10	Both parties view history	Per-booking timeline (requester + managers); global audit trail with chain verification (admin)

Extras: extension requests with conflict checks, cancellations (late cancellations tracked), training-request flow, notification centre (approval, rejection, waitlist, reminder, no-show, overdue, incident, appeal).

6. Features completed in the hackathon

Completed: all 10 lifecycle steps; four roles (user, owner, manager, admin) with server-side permission checks; identity verification gate; certification/training gate with request-and-grant flow; explainable smart search; eligibility checks; fair-use quotas + priority scoring; waiting list with auto-promotion; conflict prevention; supervised check-in; before/after condition records; auto-incidents + assistive summaries; appeals; simulated deposits; notifications; background sweep (reminders, no-shows, overdue); hash-chained audit log + verifier; fair-use watch; responsive, keyboard-accessible UI with light/dark theme; 60-check automated test suite.

Not completed / simplified (honest list):

- Identity verification is a manual admin action; no SSO/registry or document upload.
- Deposits are simulated; no payment gateway.
- Notifications are in-app only (no email/SMS/push).
- No photo evidence upload for condition reports (text + rating only).
- Single institution time zone; no calendar (iCal) sync; no recurring bookings.
- Data resets on restart on free hosting tiers (SQLite file on ephemeral disk).

7. Technology stack and architecture

```
Browser (vanilla JS SPA, no build)  <--JSON/HTTPS-->  Node.js 22 HTTP server (server.js)
                                                        |-- rules engine: eligibility,
fairness, conflicts, state machine
                                                        |-- sweep job (30 s): reminders,
no-shows, overdue
                                                        '-- SQLite (node:sqlite): users,
resources, bookings, incidents, audit (hash chain)
```

- **Backend:** Node.js 22 built-ins only (`http`, `node:sqlite`, `crypto`) - **zero third-party dependencies**, nothing to install, no supply-chain risk, trivial to deploy.
- **Frontend:** plain HTML/CSS/JS; every dynamic value is HTML-escaped; strict Content-Security-Policy.
- **Data model:** `users, resources, resource_managers, user_certs, cert_requests, bookings, extensions, incidents, notifications, audit`.

- **State machine (booking):** `pending/waitlisted -> approved -> checked_in -> returned -> completed`, with `rejected`, `cancelled`, `no_show` as exits.
- **Time handling:** all times stored as UTC epoch ms and evaluated in the institution time zone (opening hours, weekly quotas) using `Intl`.

Fairness algorithm (shown to users and owners)

`priority = 50 + purpose (coursework +15, research +10, project +5) + 10 if first time on this resource - min(30, 1.5 x hours used on this resource in the last 30 days) - 15 per strike + min(15, 2 x days waiting)`. It **only orders** competing/waitlisted requests; hard limits (weekly quota, open-request cap, max duration) are enforced separately; the owner makes the final decision.

AI tools, APIs, frameworks and external services

- **No external API or AI service is called at runtime.** "Smart matching" is a transparent synonym/keyword scorer that lists why each result matched. Incident "assistive summaries" are template-generated and explicitly labelled *not a decision*.
- **Development tools:** [DISCLOSE here the AI assistants / tools your team used, e.g. "Claude was used to scaffold and review code; the team reviewed, tested and modified it"]. Significant AI use must be disclosed under rule 5.

8. Privacy, security and fairness considerations

Privacy: demo data is fictional. Managers see a limited requester view (name, member type, organisation, status, strikes, relevant certifications); **email/contact is revealed only after approval**. ID numbers are masked in the admin UI. Users can only open their own bookings; the global audit trail is admin-only. **Security:** scrypt-hashed passwords; random session tokens (7-day expiry); login rate limiting; all permissions enforced server-side (never UI-only); parameterised SQL; request size limit; CSP, X-Frame-Options, nosniff headers; output escaping against XSS. **Accountability:** tamper-evident audit log (each entry hashes the previous), mandatory reasons on adverse decisions, penalties never automatic, independent admin appeal. **Fairness & accessibility:** quotas, caps, explainable priority, transparent waiting list, plain-language reasons; semantic HTML, keyboard focus styles, skip link, ARIA live regions, colour contrast with status text (not colour alone), responsive down to phone width.

9. Known limitations

See "Not completed" in section 6. Also: the priority score weights are illustrative and should be tuned with each institution's policy; the audit chain proves integrity of the stored history but anchoring the head hash externally would be needed against a malicious database administrator.

10. Future development

Institutional SSO and ID-document verification; photo evidence at check-in/out; email/WhatsApp notifications; payments/credits and sponsored-access wallets; multi-institution federation; calendar sync; QR check-in; ML-

based anomaly detection and an LLM drafting assistant for incident summaries (human-reviewed); usage analytics to show idle-hour recovery per resource.

11. Testing instructions for judges

1. Open **[PUBLIC URL]** (first load may take ~30 s if the host was asleep).
2. Click any **one-click demo login** on the sign-in page. Password for all accounts: `demo1234`.
3. Follow the **Judge guide** page (top navigation) - a 10-step journey in about 5 minutes. Shortest complete path:
    - *Ananya* -> Discover -> **Podcast Studio Room** -> Check my request -> Submit (auto-approved) -> Check in -> Return.
    - *Dr. Kavita Nair* (owner) -> Manage -> In progress -> Confirm return.
    - *Dr. Meera Iyer* (admin) -> Admin -> Audit trail -> Verify chain.
4. Full approval path: *Vikram* requests the **Camera Kit** -> *Dr. Kavita Nair* approves it from Manage. Training gate: Vikram -> CNC router -> Request training -> *Rohan Das* grants it. Waitlist: *Ananya* cancels tomorrow's laser booking -> *Ishaan* is promoted -> *Rohan* approves and checks him in.
5. Press **Reset demo data** (Admin > Overview) to restore the starting state.
6. Automated tests: `node test/smoke.js` (60 checks).

12. Team and contributions

Name	Role	Email	Personal contribution (be specific and truthful)
[Member 1]	[e.g. Backend]		
[Member 2]	[e.g. Frontend]		
[Member 3]	[e.g. UI/UX / Product / Presentation]		

Every member must also submit the form individually with their own role and what they personally built.

13. Disclosure of templates, libraries and pre-existing code

No templates or third-party libraries. Built on Node.js 22 standard library and browser APIs. No pre-existing project code. AI assistance: [DESCRIBE]. All names, organisations and equipment are fictional.